



Pontifícia Universidade Católica do Rio Grande do Sul
Confiabilidade e Segurança de Software
98G08-04

Lista de Exercícios
Cifras de Bloco
(Com Respostas)

Prof. Iaçanã Ianiski Weber

Observação: esta versão contém os enunciados originais e uma proposta de solução. Em itens discursivos, respostas equivalentes podem ser igualmente corretas.

Tabela de inversos multiplicativos em $GF(2^8)$ usados no AES, com

$$P(x) = x^8 + x^4 + x^3 + x + 1.$$

$X \backslash Y$	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	00	01	8D	F6	CB	52	7B	D1	E8	4F	29	C0	B0	E1	E5	C7
1	74	B4	AA	4B	99	2B	60	5F	58	3F	FD	CC	FF	40	EE	B2
2	3A	6E	5A	F1	55	4D	A8	C9	C1	0A	98	15	30	44	A2	C2
3	2C	45	92	6C	F3	39	66	42	F2	35	20	6F	77	BB	59	19
4	1D	FE	37	67	2D	31	F5	69	A7	64	AB	13	54	25	E9	09
5	ED	5C	05	CA	4C	24	87	BF	18	3E	22	F0	51	EC	61	17
6	16	5E	AF	D3	49	A6	36	43	F4	47	91	DF	33	93	21	3B
7	79	B7	97	85	10	B5	BA	3C	B6	70	D0	06	A1	FA	81	82
8	83	7E	7F	80	96	73	BE	56	9B	9E	95	D9	F7	02	B9	A4
9	DE	6A	32	6D	D8	8A	84	72	2A	14	9F	88	F9	DC	89	9A
A	FB	7C	2E	C3	8F	B8	65	48	26	C8	12	4A	CE	E7	D2	62
B	0C	E0	1F	EF	11	75	78	71	A5	8E	76	3D	BD	BC	86	57
C	0B	28	2F	A3	DA	D4	E4	0F	A9	27	53	04	1B	FC	AC	E6
D	7A	07	AE	63	C5	DB	E2	EA	94	8B	C4	D5	9D	F8	90	6B
E	B1	0D	D6	EB	C6	0E	CF	AD	08	4E	D7	E3	5D	50	1E	B3
F	5B	23	38	34	68	46	03	8C	DD	9C	7D	A0	CD	1A	41	1C

Tabela da S-box do AES: valores de substituição em notação hexadecimal para o byte de entrada (xy) .

$x \backslash y$	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

Exercício 1

Calcule $A(x) + B(x) \bmod P(x)$ em $\text{GF}(2^4)$ usando $P(x) = x^4 + x + 1$. O que aconteceria se alterássemos o polinômio de redução?

1. $A(x) = x^2 + 1, \quad B(x) = x^3 + x^2 + 1.$
2. $A(x) = x^2 + 1, \quad B(x) = x + 1.$

A soma em $\text{GF}(2^n)$ é simplesmente o XOR coeficiente a coeficiente. Como $\deg A, \deg B < 4$, o resultado já tem grau < 4 e *nenhuma* redução por $P(x)$ é aplicada.

1. $A(x) + B(x) = (x^2 + 1) \oplus (x^3 + x^2 + 1) = x^3$. Em binário: 1000, ou seja 0x8.
2. $A(x) + B(x) = (x^2 + 1) \oplus (x + 1) = x^2 + x$. Em binário: 0110, ou seja 0x6.

Influência de $P(x)$: a soma nunca aumenta o grau, então a redução por $P(x)$ nunca é acionada e o resultado independe da escolha do polinômio irredutível.

Exercício 2

Calcule $A(x) \cdot B(x) \bmod P(x)$ em $\text{GF}(2^4)$ usando $P(x) = x^4 + x + 1$. Qual a influência da escolha do polinômio de redução?

1. $A(x) = x^2 + 1, \quad B(x) = x^3 + x^2 + 1.$
2. $A(x) = x^2 + 1, \quad B(x) = x + 1.$

1. $(x^2 + 1)(x^3 + x^2 + 1) = x^5 + x^4 + x^3 + 1$ (em $\text{GF}(2)$, $2x^2 = 0$).

Reduzindo módulo $P(x) = x^4 + x + 1$, temos $x^4 \equiv x + 1$ e $x^5 = x \cdot x^4 \equiv x^2 + x$. Logo

$$x^5 + x^4 + x^3 + 1 \equiv (x^2 + x) + (x + 1) + x^3 + 1 = x^3 + x^2.$$

Em binário: 1100, ou seja 0xC.

2. $(x^2 + 1)(x + 1) = x^3 + x^2 + x + 1$. Como $\deg = 3 < 4$, não há redução. Em binário: 1111, ou seja 0xF.

Influência de $P(x)$: ao contrário da soma, a multiplicação produz polinômios de grau até $2(n-1)$ e *precisa* ser reduzida módulo $P(x)$. Polinômios irredutíveis distintos geram corpos isomorfos, mas com tabelas de multiplicação numericamente diferentes.

Exercício 3

Calcule em $\text{GF}(2^8)$:

$$\frac{x^4 + x + 1}{x^7 + x^6 + x^3 + x^2}$$

com $P(x) = x^8 + x^4 + x^3 + x + 1$.

Em hexadecimal: numerador 0x13, denominador 0xCC.

Divisão = multiplicação pelo inverso. Pela tabela de inversos, linha C, coluna C:

$$(0xCC)^{-1} = 0x1B.$$

Resta computar $0x13 \cdot 0x1B$ em $\text{GF}(2^8)$.

$$(x^4 + x + 1)(x^4 + x^3 + x + 1) = x^8 + x^7 + x^4 + x^3 + x^2 + 1.$$

Aplicando $x^8 \equiv x^4 + x^3 + x + 1$:

$$(x^4 + x^3 + x + 1) + x^7 + x^4 + x^3 + x^2 + 1 = x^7 + x^2 + x.$$

Em binário: 1000 0110, ou seja 0x86.

Exercício 4

Considere a primeira parte da operação ByteSub: a inversão em $\text{GF}(2^8)$.

1. Determine os inversos dos bytes 29, F3 e 01.
2. Verifique multiplicando em $\text{GF}(2^8)$.

Da tabela de inversos:

$$(0x29)^{-1} = 0x0A, \quad (0xF3)^{-1} = 0x34, \quad (0x01)^{-1} = 0x01.$$

Verificação de $0x29 \cdot 0x0A$:

$$0x29 = x^5 + x^3 + 1, \quad 0x0A = x^3 + x.$$

$$(x^5 + x^3 + 1)(x^3 + x) = x^8 + x^4 + x^3 + x.$$

Aplicando $x^8 \equiv x^4 + x^3 + x + 1$:

$$(x^4 + x^3 + x + 1) + x^4 + x^3 + x = 1. \quad \checkmark$$

Verificação de $0xF3 \cdot 0x34$: substituindo $0xF3 = x^7 + x^6 + x^5 + x^4 + x + 1$ e $0x34 = x^5 + x^4 + x^2$, expandindo e reduzindo módulo $P(x)$, obtém-se 1.

Verificação de $0x01 \cdot 0x01 = 1$ é trivial.

Exercício 5

Calcule a S-box (operação ByteSub) para os bytes 29, F3 e 01.

1. Use a tabela de inversos para obter B' e em seguida aplique o mapeamento afim.
2. Confirme com a tabela da S-box.
3. Qual o valor de $S(0)$?

A operação ByteSub é $S(B) = A \cdot B^{-1} + c$, onde a transformação afim é dada pela matriz A (8×8 sobre $\text{GF}(2)$) e o vetor constante $c = 0x63$.

Equivalentemente, escrevendo cada bit:

$$b'_i = b_i \oplus b_{(i+4) \bmod 8} \oplus b_{(i+5) \bmod 8} \oplus b_{(i+6) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus c_i,$$

onde b é o inverso (já calculado no Ex. 4) e c_i é o i -ésimo bit de $0x63 = 0110\,0011$.

Para $0x29$: $B' = 0x0A$. Aplicando a transformação afim, obtém-se $0xA5$.

Para $0xF3$: $B' = 0x34$. Aplicando a transformação afim, obtém-se $0x0D$.

Para $0x01$: $B' = 0x01$. Aplicando a transformação afim, obtém-se $0x7C$.

Conferência pela tabela da S-box (linha X , coluna Y do byte $0xXY$):

$$S(0x29) = 0xA5, \quad S(0xF3) = 0x0D, \quad S(0x01) = 0x7C. \quad \checkmark$$

Valor de $S(0)$: por convenção define-se $0^{-1} := 0$ (o 0 não tem inverso). Aplicando a parte afim a $0x00$, sobra apenas o vetor constante:

$$S(0) = c = 0x63.$$

Exercício 6

Qual a saída da primeira rodada do AES-128 se a entrada da camada SubBytes é toda 1 (128 bits) e a subchave k_1 também é toda 1?

A entrada de SubBytes é uma matriz 4×4 com todos os bytes iguais a 0xFF.

SubBytes: $S(0xFF) = 0x16$ (linha F, coluna F). Logo, todos os 16 bytes do estado passam a 0x16.

ShiftRows: desloca linhas, mas como todos os bytes são iguais, o estado não muda.

MixColumns: cada coluna é multiplicada pela matriz MDS $\begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix}$.

A soma de cada linha é $02 \oplus 03 \oplus 01 \oplus 01 = 01$. Como cada coluna do estado tem os quatro bytes iguais a v , o resultado de cada byte da coluna é $01 \cdot v = v$. Logo, o estado continua sendo 0x16 em todas as posições.

AddRoundKey com k_1 todo 0xFF:

$$0x16 \oplus 0xFF = 0xE9.$$

Estado final:

$$\begin{bmatrix} \text{E9} & \text{E9} & \text{E9} & \text{E9} \\ \text{E9} & \text{E9} & \text{E9} & \text{E9} \\ \text{E9} & \text{E9} & \text{E9} & \text{E9} \\ \text{E9} & \text{E9} & \text{E9} & \text{E9} \end{bmatrix}.$$

Exercício 7

Verifique a difusão do AES após uma única rodada com $X = (0x01000000, 0, 0, 0)$ e as subchaves $W[0] \dots W[7]$ dadas. Compare com a saída quando $X = 0$.

Organizando X como matriz de estado (*column-major*) e as chaves $K_0 = W[0..3]$ e $K_1 = W[4..7]$:

$$X = \begin{bmatrix} 01 & 00 & 00 & 00 \\ 00 & 00 & 00 & 00 \\ 00 & 00 & 00 & 00 \\ 00 & 00 & 00 & 00 \end{bmatrix}, K_0 = \begin{bmatrix} 2B & 28 & AB & 09 \\ 7E & AE & F7 & CF \\ 15 & D2 & 15 & 4F \\ 16 & A6 & 88 & 3C \end{bmatrix}, K_1 = \begin{bmatrix} A0 & 88 & 23 & 2A \\ FA & 54 & A3 & 6C \\ FE & 2C & 39 & 76 \\ 17 & B1 & 39 & 05 \end{bmatrix}.$$

Caso (1): entrada com 1 bit ativo.

Após ARK(K_0): $\begin{bmatrix} 2A & 28 & AB & 09 \\ 7E & AE & F7 & CF \\ 15 & D2 & 15 & 4F \\ 16 & A6 & 88 & 3C \end{bmatrix}$.

Após SubBytes (lookup S-box): $\begin{bmatrix} E5 & 34 & 62 & 01 \\ F3 & E4 & 68 & 8A \\ 59 & B5 & 59 & 84 \\ 47 & 24 & C4 & EB \end{bmatrix}$.

Após ShiftRows (linha i desloca i posições à esquerda): $\begin{bmatrix} \text{E5} & \text{34} & \text{62} & \text{01} \\ \text{E4} & \text{68} & \text{8A} & \text{F3} \\ \text{59} & \text{84} & \text{59} & \text{B5} \\ \text{EB} & \text{47} & \text{24} & \text{C4} \end{bmatrix}$.

Após MixColumns (multiplicação coluna-a-coluna por MDS): $\begin{bmatrix} \text{54} & \text{13} & \text{3C} & \text{7D} \\ \text{36} & \text{34} & \text{A2} & \text{FC} \\ \text{95} & \text{86} & \text{36} & \text{D4} \\ \text{44} & \text{3E} & \text{3D} & \text{D6} \end{bmatrix}$.

Após $\text{ARK}(K_1)$ (saída da rodada 1 para X com 1 bit):

$$Y_1 = \begin{bmatrix} \text{F4} & \text{9B} & \text{1F} & \text{57} \\ \text{CC} & \text{60} & \text{01} & \text{90} \\ \text{6B} & \text{AA} & \text{0F} & \text{A2} \\ \text{53} & \text{8F} & \text{04} & \text{D3} \end{bmatrix}.$$

Caso (2): entrada $X = 0$.

Repetindo as mesmas etapas com estado inicial nulo (somente K_0 entra em SubBytes), as colunas 1, 2, 3 de saída são idênticas ao caso (1) (pois MixColumns é coluna-a-coluna e só a coluna 0 depende do bit alterado). A coluna 0 muda. Resultado:

$$Y_0 = \begin{bmatrix} \text{DC} & \text{9B} & \text{1F} & \text{57} \\ \text{D8} & \text{60} & \text{01} & \text{90} \\ \text{7F} & \text{AA} & \text{0F} & \text{A2} \\ \text{6F} & \text{8F} & \text{04} & \text{D3} \end{bmatrix}.$$

Diferença $Y_1 \oplus Y_0$:

$$\begin{bmatrix} \text{28} & \text{0} & \text{0} & \text{0} \\ \text{14} & \text{0} & \text{0} & \text{0} \\ \text{14} & \text{0} & \text{0} & \text{0} \\ \text{3C} & \text{0} & \text{0} & \text{0} \end{bmatrix} \implies \text{bits diferentes} = 2 + 2 + 2 + 4 = \boxed{10 \text{ bits}}.$$

Comentário (efeito avalanche): a diferença de 1 bit na entrada produz, após apenas *uma* rodada, alterações em 4 bytes (uma coluna inteira) em ~ 10 bits dos 128 totais. Após cada rodada subsequente, MixColumns + ShiftRows espalham essas diferenças para todas as posições; o resultado típico depois de poucas rodadas é $\approx 50\%$ dos bits alterados.

Exercício 8

Estamos na 9ª rodada de uma decifragem AES-128. Calcule a rodada final e o texto em claro, dada a chave k_0 e o estado após a substituição inversa de bytes na rodada 9.

A 9ª rodada da decifragem (penúltima) está pela metade: o estado dado é o resultado de *InvByteSub* dessa rodada. A partir dele, ainda falta:

- (a) $\text{ARK}(k_1)$ (fim da rodada 9 da decifragem);

(b) rodada final da decifragem: $InvMixColumns \rightarrow InvShiftRows \rightarrow InvByteSub$;

(c) $ARK(k_0)$ (*whitening* inicial), produzindo o texto em claro.

Subchave k_1 (obtida pelo Key Schedule do AES-128 a partir de k_0):

$$k_1 = \begin{bmatrix} \text{C2} & 86 & 0\text{E} & 2\text{C} \\ 0\text{A} & 5\text{F} & \text{C6} & \text{F5} \\ \text{DE} & \text{B8} & \text{B8} & \text{FC} \\ \text{A0} & \text{D7} & \text{C6} & 93 \end{bmatrix}.$$

(a) Após $ARK(k_1)$:

$$\begin{bmatrix} 77 & \text{e9} & 45 & 58 \\ \text{c4} & \text{cf} & \text{fa} & \text{c3} \\ \text{b8} & 66 & 87 & 95 \\ 05 & 23 & 31 & 20 \end{bmatrix} \oplus k_1 = \begin{bmatrix} \text{B5} & 6\text{F} & 4\text{B} & 74 \\ \text{CE} & 90 & 3\text{C} & 36 \\ 66 & \text{DE} & 3\text{F} & 69 \\ \text{A5} & \text{F4} & \text{F7} & \text{B3} \end{bmatrix}.$$

(b1) Após *InvMixColumns*:

$$\begin{bmatrix} 2\text{D} & 5\text{F} & 5\text{E} & \text{B0} \\ 3\text{D} & \text{A0} & 0\text{B} & 9\text{E} \\ 95 & 5\text{F} & 4\text{F} & 86 \\ 3\text{D} & 75 & \text{A5} & 30 \end{bmatrix}.$$

(b2) Após *InvShiftRows* (linha i desloca i posições à direita):

$$\begin{bmatrix} 2\text{D} & 5\text{F} & 5\text{E} & \text{B0} \\ 9\text{E} & 3\text{D} & \text{A0} & 0\text{B} \\ 4\text{F} & 86 & 95 & 5\text{F} \\ 75 & \text{A5} & 30 & 3\text{D} \end{bmatrix}.$$

(b3) Após *InvByteSub* (S-box inversa):

$$\begin{bmatrix} \text{FA} & 84 & 9\text{D} & \text{FC} \\ \text{DF} & 8\text{B} & 47 & 9\text{E} \\ 92 & \text{DC} & \text{AD} & 84 \\ 3\text{F} & 29 & 08 & 8\text{B} \end{bmatrix}.$$

(c) Após $ARK(k_0)$ — **texto em claro**:

$$\begin{bmatrix} \text{FA} & \text{C0} & 15 & \text{DE} \\ \text{CE} & \text{DE} & \text{DE} & \text{AD} \\ \text{B0} & \text{BA} & \text{AD} & \text{C0} \\ 0\text{C} & 5\text{E} & 19 & \text{DE} \end{bmatrix}$$

Texto em claro (lendo em ordem *column-major*, como aparece no canal):

FA CE B0 0C C0 DE BA 5E 15 DE AD 19 DE AD C0 DE.

Exercício 9

AES-192 com $3 \cdot 10^7$ chaves/s por chip e 10^5 chips em paralelo.

1. Tempo médio de busca exaustiva. Compare com a idade do universo ($\approx 10^{10}$ anos).
2. Quantos anos teremos que esperar (Lei de Moore) até conseguir 24 h de busca?

Item 1. Velocidade total: $10^5 \cdot 3 \cdot 10^7 = 3 \cdot 10^{12}$ chaves/s. Em média, a busca exige 2^{191} tentativas (metade do espaço de 2^{192} chaves).

$$T = \frac{2^{191}}{3 \cdot 10^{12}} \approx \frac{3,14 \cdot 10^{57}}{3 \cdot 10^{12}} \approx 1,05 \cdot 10^{45} \text{ s.}$$

Convertendo para anos ($1 \text{ ano} \approx 3,15 \cdot 10^7 \text{ s}$):

$$T \approx 3,3 \cdot 10^{37} \text{ anos.}$$

Comparando com a idade do universo ($\approx 10^{10}$ anos):

$$\frac{T}{T_{\text{universo}}} \approx 3,3 \cdot 10^{27}.$$

Ou seja, a busca leva da ordem de 10^{27} idades do universo. Inviável.

Item 2. Lei de Moore: a velocidade dobra a cada 1,5 ano. Queremos $T = 24 \text{ h} = 86400 \text{ s}$:

$$\text{velocidade necessária} = \frac{2^{191}}{86400} \approx 3,66 \cdot 10^{52} \text{ chaves/s,}$$

distribuída em 10^5 chips, dá $\approx 3,66 \cdot 10^{47}$ chaves/s por chip.

Speedup necessário sobre o atual ($3 \cdot 10^7$ s/chip):

$$\text{speedup} = \frac{3,66 \cdot 10^{47}}{3 \cdot 10^7} \approx 1,22 \cdot 10^{40} \approx 2^{133}.$$

Tempo até atingir esse speedup:

$$2^{t/1,5} = 2^{133} \implies t = 1,5 \cdot 133 \approx \boxed{200 \text{ anos}}.$$

(Pressupondo, naturalmente, que a Lei de Moore continue válida por dois séculos — o que está longe de ser garantido.)

Exercício 10

Cifra didática $e(b_1b_2b_3b_4b_5) = (b_2b_5b_4b_1b_3)$. Encripte $x = 01101\,11011\,11010\,00110$ em ECB, CBC, CFB, OFB e CTR, com $IV = 11001$.

Blocos de entrada: $x_1 = 01101$, $x_2 = 11011$, $x_3 = 11010$, $x_4 = 00110$.

A função e é uma permutação dos cinco bits. Aplicando-a aos valores que aparecem nos cálculos abaixo:

x	$e(x)$	x	$e(x)$	x	$e(x)$
01101	11001	11001	11010	10100	00011
11011	11110	11000	10010	01000	10000
11010	10110	10110	00111	00111	01101
00110	00101	10111	01111	11100	10011

ECB. $y_i = e(x_i)$:

$$y = 11001\,11110\,10110\,00101.$$

CBC. $y_1 = e(x_1 \oplus IV)$ e $y_i = e(x_i \oplus y_{i-1})$:

$$y_1 = e(01101 \oplus 11001) = e(10100) = 00011,$$

$$y_2 = e(11011 \oplus 00011) = e(11000) = 10010,$$

$$y_3 = e(11010 \oplus 10010) = e(01000) = 10000,$$

$$y_4 = e(00110 \oplus 10000) = e(10110) = 00111.$$

$$y = 00011\,10010\,10000\,00111.$$

OFB. $s_1 = e(IV)$, $s_i = e(s_{i-1})$, $y_i = s_i \oplus x_i$:

$$s_1 = e(11001) = 11010, \quad y_1 = 11010 \oplus 01101 = 10111,$$

$$s_2 = e(11010) = 10110, \quad y_2 = 10110 \oplus 11011 = 01101,$$

$$s_3 = e(10110) = 00111, \quad y_3 = 00111 \oplus 11010 = 11101,$$

$$s_4 = e(00111) = 01101, \quad y_4 = 01101 \oplus 00110 = 01011.$$

$$y = 10111\,01101\,11101\,01011.$$

CFB. $y_1 = e(IV) \oplus x_1$ e $y_i = e(y_{i-1}) \oplus x_i$:

$$\begin{aligned}y_1 &= e(11001) \oplus 01101 = 11010 \oplus 01101 = 10111, \\y_2 &= e(10111) \oplus 11011 = 01111 \oplus 11011 = 10100, \\y_3 &= e(10100) \oplus 11010 = 00011 \oplus 11010 = 11001, \\y_4 &= e(11001) \oplus 00110 = 11010 \oplus 00110 = 11100.\end{aligned}$$

$$y = 10111 \ 10100 \ 11001 \ 11100.$$

CTR. Contador iniciado em IV e incrementado: $CTR_i = IV + i - 1$.

$$\begin{array}{lll}CTR_1 = 11001, & e(CTR_1) = 11010, & y_1 = 11010 \oplus 01101 = 10111, \\CTR_2 = 11010, & e(CTR_2) = 10110, & y_2 = 10110 \oplus 11011 = 01101, \\CTR_3 = 11011, & e(CTR_3) = 11110, & y_3 = 11110 \oplus 11010 = 00100, \\CTR_4 = 11100, & e(CTR_4) = 10011, & y_4 = 10011 \oplus 00110 = 10101.\end{array}$$

$$y = 10111 \ 01101 \ 00100 \ 10101.$$

Exercício 11

Em uma rede interna, AES-128 em CBC com chave fixa e IV diário. Você descobriu a chave mas não conhece o IV. Hoje interceptou dois arquivos: um desconhecido e outro contendo apenas 0xFF (16 bytes). Como recuperar o IV e decifrar o desconhecido?

Recuperação do IV. Seja $y_1^{(\text{FF})}$ o primeiro bloco cifrado do arquivo conhecido (todo 0xFF) e $x_1^{(\text{FF})} = \text{FF FF} \dots \text{FF}$ (16 bytes). Pela definição do modo CBC,

$$y_1^{(\text{FF})} = e_k(x_1^{(\text{FF})} \oplus IV) \implies e_k^{-1}(y_1^{(\text{FF})}) = x_1^{(\text{FF})} \oplus IV.$$

Como o atacante conhece k , $y_1^{(\text{FF})}$ e $x_1^{(\text{FF})}$:

$$IV = e_k^{-1}(y_1^{(\text{FF})}) \oplus x_1^{(\text{FF})}.$$

Decifragem do arquivo desconhecido. Seja $y_1^{(?)}, y_2^{(?)}, \dots$ o arquivo desconhecido. Aplicando a decifragem CBC padrão com o IV recuperado:

$$x_1^{(?)} = e_k^{-1}(y_1^{(?)}) \oplus IV, \quad x_i^{(?)} = e_k^{-1}(y_i^{(?)}) \oplus y_{i-1}^{(?)} \quad (i \geq 2).$$

Observação prática: esse ataque depende de termos um par (x_1, y_1) conhecido, o que ocorre frequentemente em sistemas que injetam arquivos temporários previsíveis na rede. É um lembrete de que *IV no CBC não pode ser apenas público — ele precisa ser único a cada arquivo*, idealmente um *nonce* aleatório, não um valor fixo por dia.

Exercício 12

Como o modo OFB pode ser atacado se o IV não for diferente a cada execução?

No OFB, a *keystream* é gerada *exclusivamente* a partir de k e IV — **não depende do texto em claro**. Ou seja, com k e IV fixos:

$$s_1 = e_k(IV), s_2 = e_k(s_1), \dots$$

é sempre a *mesma* sequência.

Logo, se duas mensagens x e x' são cifradas com o mesmo par (k, IV) :

$$y_i = s_i \oplus x_i, \quad y'_i = s_i \oplus x'_i.$$

Aplicando XOR:

$$y_i \oplus y'_i = x_i \oplus x'_i.$$

A *keystream* *cancela*, e o atacante obtém o XOR dos textos em claro. Esse é o ataque clássico de *many-time pad*.

Exploração com plaintext conhecido (a dica do enunciado). Muitos formatos de arquivo começam com *magic numbers* fixos: PNG inicia com 89 50 4E 47 0D 0A 1A 0A, ZIP com 50 4B 03 04, PDF com %PDF-, JPEG com FF D8 FF, etc.

Se o atacante *suspeita* que x é um PNG, ele toma o cabeçalho conhecido x_{hdr} e calcula

$$x'_{\text{hdr}} = (y \oplus y') \oplus x_{\text{hdr}}.$$

Os bytes de x' correspondentes ao cabeçalho são revelados imediatamente. A partir daí, com texto e estatística (n-gramas, dicionário, padrões), normalmente é possível recuperar boa parte de ambas as mensagens.

Pior caso: se x_i é totalmente conhecido em algum trecho, então para esse trecho o atacante recupera $s_i = y_i \oplus x_i$ — e qualquer outra mensagem cifrada com o mesmo (k, IV) é decifrada *naquele trecho* sem esforço.

Conclusão: no OFB, IV deve ser *único por execução (nonce)*. Reusar IV é equivalente a reusar uma chave de *one-time pad* — destrói a confidencialidade.

Exercício 13

AES no modo CTR para encriptar 1 TB. Qual o comprimento máximo do IV?

AES tem bloco de 128 bits = 16 bytes. O input da cifra no modo CTR é o par $(IV \parallel \text{CTR})$, totalizando 128 bits.

Quantidade de blocos para cobrir 1 TB.

$$\frac{1 \text{ TB}}{16 \text{ bytes}} = \frac{2^{40} \text{ bytes}}{2^4 \text{ bytes}} = 2^{36} \text{ blocos.}$$

(Adotando o convênio binário $1 \text{ TB} = 2^{40} \text{ bytes}$.)

Tamanho mínimo do contador. Para que cada bloco tenha um valor de contador único, precisamos de pelo menos 36 bits no contador.

Tamanho máximo do IV.

$$|IV|_{\max} = 128 - 36 = 92 \text{ bits.}$$

(Se for usado $1 \text{ TB} = 10^{12} \text{ bytes}$, o número de blocos é $\approx 6,25 \cdot 10^{10}$, ainda exigindo 36 bits de contador, e a resposta é a mesma.)

Exercício 14

No CFB com largura de realimentação de 1 bit, mostre que a sincronização é restaurada após $k + 1$ passos depois de uma deleção/inserção, onde k é o tamanho do bloco da cifra.

Configuração. No CFB com realimentação de 1 bit, o registrador de deslocamento (*shift register*, SR) tem k bits. Em cada passo:

- (a) o emissor calcula $z_t = \text{MSB}(e_k(SR_t))$ e $c_t = p_t \oplus z_t$;
- (b) desloca c_t para dentro do SR (descartando o bit mais antigo).

O receptor faz o mesmo: usa o SR atual para gerar z_t , recupera $p_t = c_t \oplus z_t$, e desloca c_t no seu SR. **Para que a decifragem seja correta, o SR do receptor precisa coincidir com o do emissor no instante de cada bit.**

Efeito de uma única deleção. Suponha que o bit c_d enviado pelo emissor seja *perdido* no canal. A partir desse ponto, o receptor passa a tratar o que recebe como se fossem os bits do emissor, mas com índices defasados em 1: quando o emissor envia c_{d+j} , o receptor o recebe como se fosse c_{d+j-1} .

Conteúdo do SR após a deleção. O SR do receptor, no momento em que ele processa o que ele *acha* ser o bit c_{d+j-1} , contém os k bits mais recentes *efetivamente recebidos* antes desse:

$$SR_{\text{rec}} = (c_{d-k+1}, \dots, c_{d-1}, c_{d+1}, \dots, c_{d+j-1}).$$

Isto é, dos k bits do SR, $\max(k - j + 1, 0)$ deles são bits anteriores à deleção e os demais são bits posteriores.

Quando o SR fica 100% pós-deleção? Tão logo $j - 1 \geq k$, ou seja, $j \geq k + 1$. Nesse instante:

$$SR_{\text{rec}} = (c_{d+1}, c_{d+2}, \dots, c_{d+k}).$$

Coincidência com o SR do emissor. O emissor, ao gerar c_{d+k+1} , usa o SR

$$SR_{\text{emi}} = (c_{d+1}, c_{d+2}, \dots, c_{d+k}).$$

Logo, $SR_{\text{rec}} = SR_{\text{emi}}$ a partir desse passo. A partir daí, todo bit deslocado é o mesmo dos dois lados, e a sincronização se mantém.

Conclusão. A sincronização é restaurada após exatamente $k + 1$ bits de texto cifrado transmitidos depois da deleção (ou inserção, por argumento simétrico). Os k primeiros bits após o erro continuam sendo decifrados incorretamente, pois o SR do receptor ainda contém pelo menos um bit defasado.